Transport Services, Ports, Multiplexing and UDP

Lecture 15

Computer Networks and Distributed Programming

Where we are: the layer under your sockets

- `bind` chose a number, `connect` named one, `accept` handed back a second socket: nobody said why.
- Now: how an arriving segment finds one socket among the hundreds open on a host.
- What UDP actually is.
- What the network underneath does to your frames, which is where the next step starts.

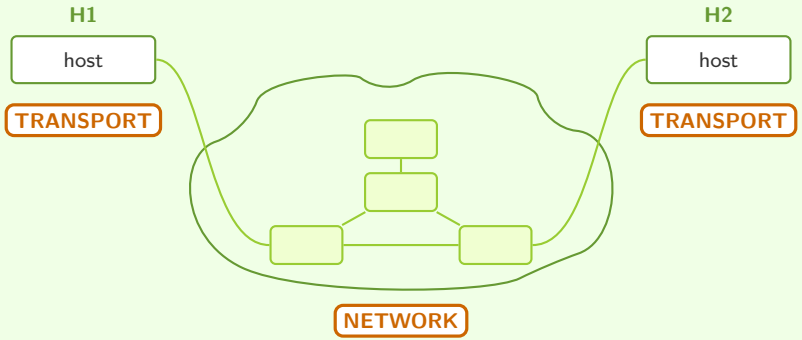
Logical communication between processes

Logical communication

From the application's point of view the two hosts look directly connected, whatever sits between them: routers, links, an ocean.

- The sending side takes an application message, possibly breaks it into chunks, and adds a header to each: a segment.
- Each segment goes down to the network layer, which encapsulates it in a datagram and makes a best-effort attempt to deliver it.
- Routers act on the fields of the datagram: what the transport layer added is not theirs to read.
- Kurose calls the transport packet a segment for both protocols; the RFCs say datagram for UDP, and so will we.

Transport at the edges, network inside



Transport protocols are implemented in the end systems and nowhere else. Routing depends on the topology, so the network layer must run where the topology is — on the nodes. Hence the two sentences that hold the whole layer together: transport connects processes, the network connects hosts.

What a transport protocol is asked to do

- Process-to-process delivery: the message reaches a process, not a machine. Host-to-host is IP's job, and it is not the same job.
- Integrity checking: an error-detection field in the header says whether the bits arrived as they left.
- Reliable data transfer: what one process sent arrives complete, correct and in order.
- Congestion control: no connection is allowed to swamp the links and routers it crosses.

UDP provides the first two. TCP provides all four — and the fourth is a service to the network as a whole, not to the application that pays for it.

Many processes, one interface

- On any host several processes are on the network at once: a browser, a mail client, a music player, your chat.
- And a single process may hold many conversations: your concurrent server kept one socket per connected client.
- So the transport layer does not deliver to a process. It delivers to a socket, and the process reads from there.
- Which decouples the number of conversations from the number of processes: one process, ten sockets, ten peers.

Ten sockets in one process. When a segment arrives, which one gets it?

Multiplexing and demultiplexing

Demultiplexing

Delivering the data of an arriving segment to the correct socket, chosen by reading fields of the segment header.

Multiplexing

Gathering data from the sockets of a host, wrapping each chunk in the header that will later be used to demultiplex it, and passing the segments down.

It takes two things: sockets must have unique identifiers, and every segment must carry the identifier of the one it is for. The problem is not peculiar to this layer: it returns wherever one protocol serves many above it.

The port number, seen from below

- Sixteen bits, from 0 to 65535, twice: a source port and a destination port, in the first four bytes of every transport header.
- The destination port is the demultiplexing key: it is what the receiving host reads to choose the socket.
- The source port is what the reply will be addressed to. The sender is announcing where to knock back.
- A port belongs to a socket, not to a process: one process can hold many, and a listening socket keeps its own while its children work.

Who may use which range, and who picks them, you already know: it is the same number `bind` took.

Which ports are open

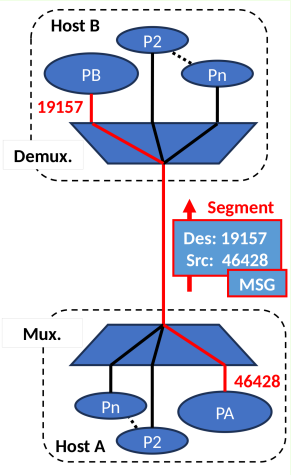
```
$ nmap --top-ports 6 scanme.nmap.org
Nmap scan report for scanme.nmap.org (45.33.32.156)
PORT     STATE    SERVICE
21/tcp   filtered ftp
22/tcp   open     ssh
23/tcp   filtered telnet
25/tcp   filtered smtp
80/tcp   open     http
110/tcp  closed   pop3
```

- `open` means something is bound and listening; `closed` means the host answered, and nothing is there.
- `filtered` means no answer came back — a firewall swallowed the probe, so the state is unknown.
- `nmap -sV` goes further and asks the port what it is, which is also the first move of whoever is looking for a way in.

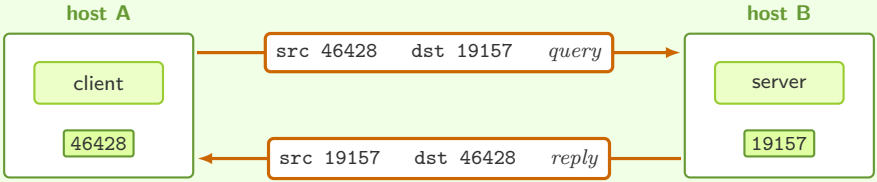
One segment, from a process to a process

A process on host A, holding port 46428, sends a message to a process on host B that is listening on 19157.

- A's transport layer builds a segment: the data, the source port, the destination port.
- It hands the segment down; the network layer wraps it and tries to deliver it to the host.
- B's transport layer strips the header, reads 19157, and puts the data in the socket that holds that port.



The source port is a return address



the source port of the first segment becomes the destination port of the second: it is the only return address the client ever sent

This is what `recvfrom` was filling in for you: the `cliaddr` you passed straight back to `sendto` is the address and port the client never had to tell you, because they travelled in the header.

A UDP socket is two numbers

Identifying a UDP socket

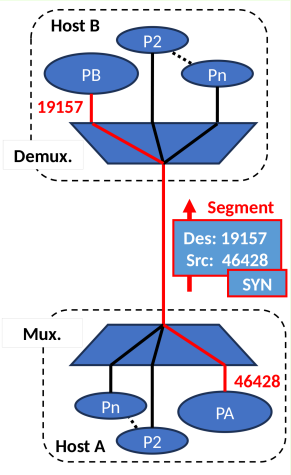
Destination IP address and destination port. Nothing else: not who sent the datagram, not from where.

- Two datagrams from two different hosts, addressed to the same pair, go to the same socket, one after the other.
- Which is why your echo server served everybody with one socket and never accepted anything.
- And why a client that disappears breaks nothing: there was no state about it to break.
- The price is that the socket cannot tell you who is talking unless `recvfrom` hands you the sender each time.

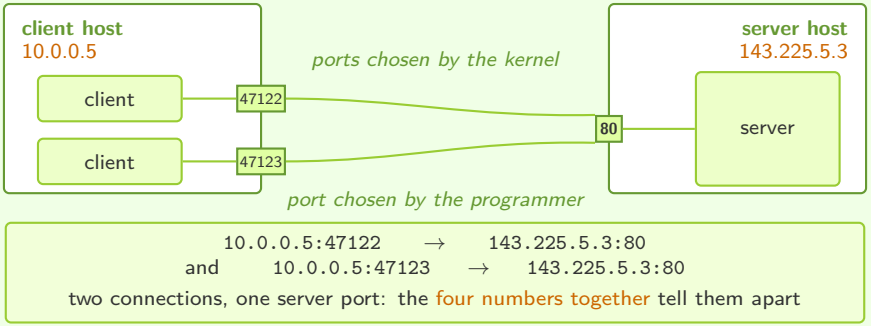
A connection request is a segment with one bit set

The server is waiting on a welcoming socket bound to 19157. The client calls `connect`.

- What goes on the wire is an ordinary segment: destination port 19157, source port picked by the kernel.
- What makes it a request is one bit in the header, the `SYN`.
- It is demultiplexed on the destination port alone: the connection it asks for does not exist yet.
- The client's ephemeral port travels in the source field, which is how the server will learn it.

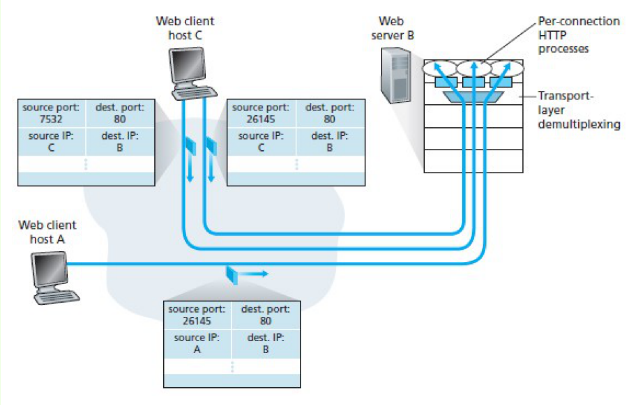


A TCP socket is four numbers



`accept` creates the second socket, and the transport layer records the four values it read in the arriving segment. Every later segment whose four values match is handed to that socket — which is why the listening one can keep listening.

Two clients, one server port



Host C gives its two sessions two source ports; host A, knowing nothing of C, picks 26145 as well. No collision: the two connections differ in the source IP — and one process per connection is only how the figure draws it.

Nobody is listening on that port

Your client sends a datagram to port 9999 of a host where nothing is bound to it. What comes back?

Nothing from the transport layer: it has no socket to deliver to, so it discards the datagram and asks the network layer to report the failure.

- The reply is an ICMP message, sent by the host and not by any process — we will read it at the network layer.
- If your socket was `connected`, the kernel can match the report and your next call fails with `ECONNREFUSED`.
- If it was not, there is nobody to hand the error to, and `recvfrom` simply waits.
- Which is why the timeout you set was not a refinement: without it the program had no way to notice.

UDP: what is left

- Two port numbers and a checksum. That is the whole protocol: multiplexing, demultiplexing, and light error detection.
- Connectionless: no handshake precedes anything. The first datagram on the wire already carries data.
- No guarantee that it arrives, no guarantee of order, no control over the rate at which you send.
- An application that chooses UDP is talking almost directly to IP, with just enough added to reach a process.

Everything TCP does beyond this, someone still has to do — above UDP, in the application.

Why an application would choose it

- Control: what you hand to UDP goes out now. TCP may hold it back to obey congestion control, or to retransmit something older.
- No setup delay: no handshake means no round trip before the first useful byte. This is the main reason DNS runs on UDP.
- No connection state: no buffers, no sequence numbers, no timers per client: the same server holds far more of them.
- Smaller header: eight bytes against twenty, on every single packet.

And reliability is still available — built on top, by the application. QUIC is build on this idea, and it is what carries the Web in HTTP/3.

A query with no handshake

- The resolver builds the query message and hands it to UDP, which adds eight bytes and passes it down.
- There is no exchange with the server first: the first packet on the wire is the question.
- Then the client waits. If no reply comes — the query was lost, or the reply was — it has three moves.
- Resend the same query; ask another name server; or tell the application that no answer arrived.

That is a typical retransmission loop, with timeout, which is necessary for a reason: nothing is going to tell you what happens to the packets.

Who uses UDP, and who cannot

- Mail, remote terminal, file transfer, the Web: no loss is tolerable, so all of them run over TCP.
- DNS: one small question, one small answer, and the delay is what the user feels. UDP.
- SNMP manages devices, and must work when the network is in trouble — exactly when reliable delivery is hardest to get. UDP.
- Multimedia: a lost sample is a click, a late one is worse than a lost one. Typically UDP, sometimes TCP.

Real-time applications react badly to congestion control: being slowed down hurts them more than losing something.

Everyone streams, nobody yields

Every video sender pushes at full rate over UDP, and none of them slows down when the network fills up. Then what?

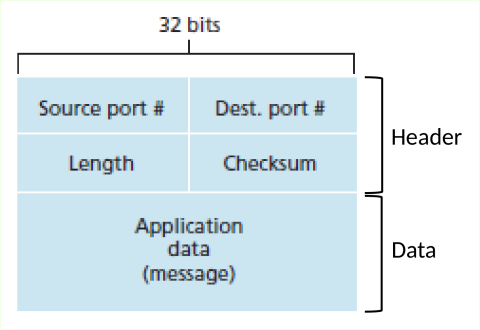
Router queues overflow, so the traffic that refuses to slow down is exactly the traffic that starts losing packets.

- And the TCP senders sharing those links do obey: they back off, and hand the capacity to those who do not.
- So the well-behaved protocol is punished for behaving, which is why running multimedia over UDP is common and controversial.
- Nothing in the network enforces restraint. It is a property of the protocols, and therefore of whoever wrote them.

The UDP segment

- Source port: where a reply should be addressed.
- Destination port: the demultiplexing key.
- Length: the whole datagram, header included, in bytes: the data field varies from one datagram to the next.
- Checksum: error detection, and nothing more.

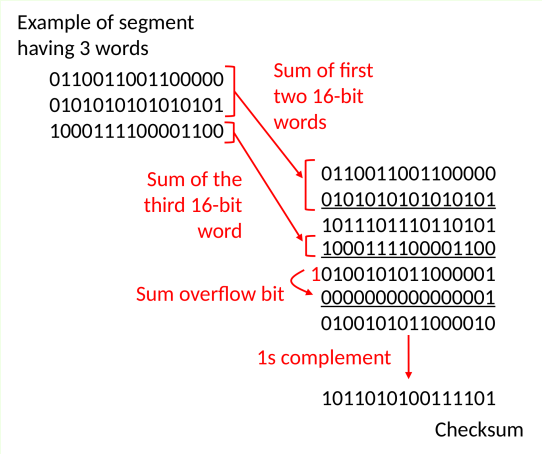
Four fields of sixteen bits: an eight-byte header, and then the application message exactly as it was handed down.



The checksum

- The sender adds all the 16-bit words of the segment, wrapping any overflow bit back into the sum.
- It takes the 1s complement of the result and puts that in the checksum field.
- The receiver adds everything, the checksum included, and expects sixteen ones.
- One zero in that sum, and the datagram is known to be damaged.

A few fields of the IP header are covered too.



Why check here, if the link layer checks

- There is no guarantee that every link along the path checks anything: IP is meant to run over whatever is there.
- And a segment can be corrupted inside a router, sitting in memory between two links that both checked it correctly.
- A function that has to hold end to end must be implemented end to end: it is the end-end principle, and it is a design rule, not a detail about UDP.
- Checking below may still be worth it — but it can never be enough on its own.

UDP detects and does not repair: the damaged datagram is dropped, or handed up with a warning, depending on the implementation.

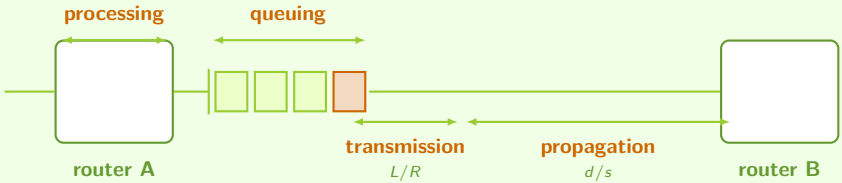
What the checksum does not catch

Two 16-bit words of the segment arrive in each other's place. Does the checksum notice?

No. Addition does not depend on order, so the sum is the same and the check passes on a segment that is wrong.

- Neither does it notice two errors that cancel out — one word too big by the amount another is too small.
- It is a cheap test that catches the common case, not a guarantee that the bits are right.
- Where errors are expected rather than exceptional — on a wire, on the air — a stronger check is needed, and we will build one at the link layer.

Four delays at every node



the first two happen inside the node, the last two on the link: only queuing changes from one packet to the next

- Processing: read the header, choose the outgoing link, check the bits: microseconds on a modern router.
- Queuing: wait behind whatever is already queued for that link.
- Transmission: push L bits into a link of rate R: the L/R you already know, now one term of four.
- Propagation: cross the distance d at speed s, a little under the speed of light: d/s.

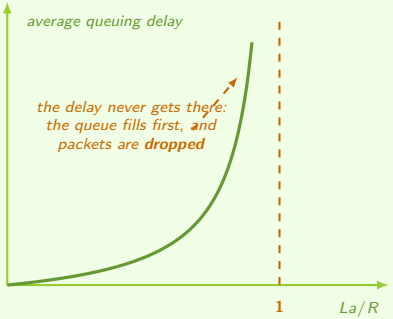
Transmission is not propagation

- Transmission depends on the packet and the rate, and not at all on how far the other end is.
- Propagation depends on the distance, and not at all on the packet or the rate.
- Ten cars leaving a tollbooth one every twelve seconds: two minutes to let them all through, one hour to drive the hundred kilometres.
- Between two routers on a campus, propagation is a couple of microseconds. Over a geostationary satellite it is hundreds of milliseconds, and it dominates everything else.

The first bits of a packet can already be arriving while the last ones are still waiting to leave.

Traffic intensity, and where loss comes from

- L bits per packet, a packets per second arriving, R bits per second leaving: the ratio is La/R.
- Above 1 the queue grows without bound. The rule of traffic engineering is to stay below it.
- Below 1 the delay depends on how bursty the arrivals are, and it climbs steeply as the ratio approaches 1.
- And a real queue is finite: one arriving at a full queue is dropped, silently.



That is where the messages your chat never received had gone.

End-to-end delay and throughput

- The nodal delays simply add up: over N links, $N\,(d_{\text{proc}} + d_{\text{trans}} + d_{\text{prop}})$, with the queuing delays on top of that.
- Throughput — what is actually flowing, as we defined it at the start of the course — is F bits received in T seconds.
- End to end, it is the rate of the slowest link on the path — the bottleneck — and adding capacity anywhere else changes nothing.
- Neither delay nor throughput is guaranteed by anything: not by IP, not by UDP, and not by TCP.

Applications that need them are built to cope, which is the only reason they work.

Summary

- The transport layer runs only on the end systems: it connects processes, while the network layer connects hosts.
- Demultiplexing is choosing the socket by reading the header; multiplexing is the same job on the way out.
- A UDP socket is identified by two numbers, a TCP socket by four — which is why one accepts and the other does not.
- UDP adds to IP only ports and a checksum, and is chosen for control, speed of setup, absence of state and a small header.
- The checksum detects and does not repair, and misses errors that cancel or reorder.
- Below it all, queues introduce delay and, when they fill, loss. Making that reliable is the next problem.

Capture a DNS query

Start a capture on your interface, type `dns` in the filter box, then run the two queries and stop it.

```
$ dig unina.it                    1.  an ordinary query
$ dig unina.it NS                 2.  who has authority
```

(a) What is the destination port of the query, and the source port of the reply? Which of the two did nobody choose?

(b) Which IP address does the query go to? Is it the resolver your machine is configured with?

(c) Which query type is it, and does the request message already carry any answer?

(d) In the reply: how many questions does it contain, and how many answers?

One question, without recursion

Before you can ask a root server anything, you have to know where it is: the first exchange is about the server, the second goes to it.

```
$ dig +short a.root-servers.net A                198.41.0.4
$ dig +norecurse @198.41.0.4 www.ox.ac.uk
$ dig @198.41.0.4 www.ox.ac.uk
```

(e) The `NS` reply of the previous capture: how many answers, how many additional records, and what are the additional ones for?

(f) Does the root server return an address or a referral? How many answer records, how many authority records?

(g) What changes on the wire between the last two commands, and who does the work in each case?

(h) Check the flags: a root server sets `aa` and never `ra`. If you see `ra`, your line is answering for the root and the whole exercise is a fake — try another network.